

✓ PA3: Convolutional Neural Networks

Introduction

In this programming assignment, you will implement a Convolutional Neural Network (CNN) in Python with the Pytorch library to classify images of handwritten digits. As part of this process, you will write your own custom Convolution and MaxPooling layers. You will then use these layers to build a LeNet-5 network. You will train your LeNet-5 with the classic MNIST dataset, which is large enough to justify training with Mini-batch gradient descent. At the end of this assignment, you will see how the LeNet-5 can achieve high accuracy on handwritten digit image classification. After that, you will tweak this model to apply it to a different image classification problem.

Objective

The main objective of this project is to practice implementing CNNs in PyTorch, including dataset loading, implementation of custom layers, training with gradient descent, and evaluation of trained models.

Instructions

Differently than the previous assignments, this time you won't be told exactly where to write your code, but will be given class and function signatures that you can implement the way you want. However, you will still have test cells with expected results, so you can check if your solution is on the right track.

✓ Part 0: Import libraries

Unlike the other practical assignments, in P4 you will not import `numpy`, but rather `torch`, to load PyTorch functionalities. You will also load the following auxiliary libraries:

- **`torch.utils.data.DataLoader`**: class for creating a data loader based on custom datasets.
- **`torchvision.datasets`**: built-in computer vision datasets.
- **`torchvision.transforms`**: set of functions for transforming images.
- **`matplotlib`**: main library for plotting graphs in Python.

```
import torch
import time
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
import matplotlib.pyplot as plt
```

✓ Part 1: Load the MNIST dataset

Implement the `load_mnist` function to load the MNIST dataset with the `datasets.MNIST()` function from the `torchvision.datasets` module. This function will return dataset objects. You will need to create one of these objects for the training set and another for the test set. You can use `transforms.ToTensor()` and `transforms.Normalize()` to convert the images to tensors and normalize them, respectively. Apply these transformations to both sets. When calling the `datasets.MNIST` function you will have to pass these two transformations into a single argument. You can use `transforms.Compose` to compose these two transformations into a single list of transformations.

After loading the train and test sets, create the `Dataloader` for each one of them. These dataloaders help you generating mini-batches at training time. Note the `batch_size` is an argument of the `load_mnist` function. Use this argument to create the dataloaders. Make sure to set `shuffle` parameters of the training dataloader to true, because we want to shuffle the data before every epoch of the mini-batch gradient descent. You don't need to shuffle the test set. Your function has to return two dataloaders, one for the training set and another for the test set.

```
def load_mnist(batch_size=32):
    # Define transformations
    transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.1307,), (0.3081,))
    ])

    # Load the training dataset
    train_dataset = datasets.MNIST('../data', train=True, download=True, transform=transform)
    # Create training DataLoader
    train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)

    # Load the test dataset
    test_dataset = datasets.MNIST('../data', train=False, download=True, transform=transform)
```

```
# Create test DataLoader
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)

return train_loader, test_loader

# Set random seed
torch.manual_seed(42)

# Load the MNIST dataset
train_loader, test_loader = load_mnist(batch_size=16)

# Print number of training and testing example
print(f"Number of training examples: {len(train_loader.dataset)}")
print(f"Number of testing examples: {len(test_loader.dataset)}")

# Get a single batch of data
data, target = next(iter(test_loader))

# Select an index to view an image
index = 10

# Plot part image
print(f"Sample pixels of image [{index}]: {data[index].numpy().squeeze()[10][:10]}")

100%|██████████| 9.91M/9.91M [00:00<00:00, 37.3MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 1.13MB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 10.2MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 11.3MB/s]
Number of training examples: 60000
Number of testing examples: 10000
Sample pixels of image [10]: [-0.42421296 -0.42421296 -0.42421296 -0.42421296 -0.42421296 -0.42421296
-0.42421296 -0.42421296  2.0450644  2.8087585 ]
```

Expected result:

```
Number of training examples  60000
Number of testing examples   10000
Sample pixels of image [10]  [-0.42421296 -0.42421296 -0.42421296 -0.42421296 -0.42421296 -0.42421296 -0.42421296 -0.42421296  2.0450644  2.8087585 ]
```

▼ Part 2: Convolutional Neural Network

In Pytorch, neural network layers and complete models are implemented extending `torch.nn.Module` class. There are two main methods you need to implement when creating a new model: `__init__` to initialize the attributes of your model and `forward`, which will perform the forward pass of your model.

2.1 A first convolution layer

The first layer you will implement is the Convolution Layer with stride and padding. The main attributes you need to consider are the number of input channels `in_channels`, the number of output channels `out_channels`, the kernel size `kernel_size`, stride size `stride` and padding size `padding`.

1. `__init__`: Create the weights (filters) and a bias vector using the `torch.nn.Parameter` function. Your filter tensor should have size `(out_channels, in_channels, kernel_size, kernel_size)`. You can initialize the weights with the `torch.randn` function. Also save the parameters values using attributes to use them in `forward`.
2. `forward(x)`: perform the forward pass on a mini-batch `x` of size `(batch_size, in_channels, in_height, in_width)`. This operation has to support padding (see `torch.nn.functional.pad`) and strides of any sizes. For this, you will have to compute the output width w_o and h_o using the formula we saw in class:

$$w_o = \frac{w_i + 2p - f}{s} + 1$$

$$h_o = \frac{h_i + 2p - f}{s} + 1$$

where w_i is the input image width, h_i is the input image height, p is the padding size, f is the filter size and s is the stride size.

With w_o and h_o computed, you can implement the convolution with 4 for loops: 1 over the batch size, 1 over the number of filters, 1 over the output height and 1 over the output width. Make sure your output tensor is of size `(batch_size, out_channels, out_height, out_width)`

```
class CustomConv2d(torch.nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size, stride=1, padding=0):
        """
        Initialize weights, bias, and hyperparameters of the convolutional layer

        Args:
```

```

        in_channels (int): number of input channels
        out_channels (int): number of output channels
        kernel_size (int): size of the kernel
        stride (int): stride size
        padding (int): padding size
    """
    super(CustomConv2d, self).__init__()
    self.in_channels = in_channels
    self.out_channels = out_channels
    self.kernel_size = kernel_size
    self.stride = stride
    self.padding = padding

    # Initialize weights (filters) of size (out_channels, in_channels, kernel_size, kernel_size)
    self.weight = torch.nn.Parameter(torch.randn(out_channels, in_channels, kernel_size, kernel_size))
    # Initialize bias of size (out_channels)
    self.bias = torch.nn.Parameter(torch.randn(out_channels))

def forward(self, x):
    """
    Perform the forward pass of a convolution layer on a mini-batch x

    Args:
        x (torch.Tensor): mini-batch of images of size (batch_size, in_channels, in_height, in_width)

    Returns:
        torch.Tensor: output of the convolutional layer of size (batch_size, out_channels, out_height, out_w
    """
    batch_size, in_channels, in_height, in_width = x.size()

    # Apply padding if necessary
    if self.padding > 0:
        x = torch.nn.functional.pad(x, (self.padding, self.padding, self.padding, self.padding))
        in_height += 2 * self.padding
        in_width += 2 * self.padding

    # Calculate output dimensions
    out_height = (in_height - self.kernel_size) // self.stride + 1
    out_width = (in_width - self.kernel_size) // self.stride + 1

    # Initialize output tensor
    output = torch.zeros(batch_size, self.out_channels, out_height, out_width)

    for b in range(batch_size):
        for c_out in range(self.out_channels):
            for h_out in range(out_height):
                for w_out in range(out_width):
                    # Calculate the receptive field
                    h_start = h_out * self.stride
                    h_end = h_start + self.kernel_size
                    w_start = w_out * self.stride
                    w_end = w_start + self.kernel_size

                    # Extract the patch from the input image
                    patch = x[b, :, h_start:h_end, w_start:w_end]

                    # Perform element-wise multiplication and sum, then add bias
                    output[b, c_out, h_out, w_out] = torch.sum(patch * self.weight[c_out]) + self.bias[c_out]

    return output

```

```

# Set random seed
torch.manual_seed(42)

# Create an instance of your custom convolution layer
custom_conv = CustomConv2d(in_channels=1, out_channels=6, kernel_size=5)

# Show output for a single example of the dataset
first_batch = next(iter(test_loader))[0]

# Pick one image in the mini-batch
first_image = first_batch[0]
output_custom_conv = custom_conv(first_image.unsqueeze(0))

# Output Sample
print("Output shape:", output_custom_conv.size())
print("Output sample:", output_custom_conv[:, :, 0, 0])

```

```

Output shape: torch.Size([1, 6, 24, 24])
Output sample: tensor([[ -0.3627,  2.7504, -2.1521,  1.8907, -2.4571,  1.1121]],
      grad_fn=<SelectBackward0>)

```

Expected result:

Output size [1, 6, 24, 24]

Output sample [[-0.3627, 2.7504, -2.1521, 1.8907, -2.4571, 1.1121]]

```
# Set random seed
torch.manual_seed(42)

# Create an instance of your custom convolution layer
custom_conv = CustomConv2d(in_channels=1, out_channels=6, kernel_size=5)

# Compare with pytorch's conv2d Layer
pytorch_conv = torch.nn.Conv2d(in_channels=1, out_channels=6, kernel_size=5)

# Forward pass of both networks
first_batch = next(iter(test_loader))[0]

# Measure time
t1 = time.perf_counter()
output_custom_conv = custom_conv(first_batch)
t2 = time.perf_counter()

print(f"Time taken by custom conv: {t2-t1} seconds")

t1 = time.perf_counter()
output_pytorch_conv = pytorch_conv(first_batch)
t2 = time.perf_counter()

print(f"Time taken by pytorch conv: {t2-t1} seconds")
```

Time taken by custom conv: 3.4023119770000108 seconds
 Time taken by pytorch conv: 0.0009533349999628626 seconds

Expected result:

Time taken by custom conv 3.8795 seconds

Time taken by pytorch conv 0.0022 seconds

2.2 Vectorizing the convolution operation

As showed in the experiment above, implementing convolutions with 4 for loops is too slow. A single mini-batch of size 16 is taking more than 3 seconds to execute, so training on the entire MNIST dataset would take approximately $15 * (60000/3) = 20000 \text{ seconds} = 5.5 \text{ hours}$. This is a very long time considering the MNIST images are only 28x28 pixels large. We can reduce this time considerably implementing the convolution operation with vectorization (as the Pytorch implementation). To do that, we need to first generate the chunks for each convolution step and then multiply them by the kernel in parallel. In the next cell, change your `CustomConv2d` cell to use vectorization. You can use Pytorch's `torch.nn.functional.unfold` function to generate the chunks for each convolution step. The `view` method of tensors might be useful here too, to reshape the tensors.

```
class CustomConv2d(torch.nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size, stride=1, padding=0):
        """
        Initialize weights, bias, and hyperparameters of the convolutional layer

        Args:
            in_channels (int): number of input channels
            out_channels (int): number of output channels
            kernel_size (int): size of the kernel
            stride (int): stride size
            padding (int): padding size
        """
        super(CustomConv2d, self).__init__()
        self.in_channels = in_channels
        self.out_channels = out_channels
        self.kernel_size = kernel_size
        self.stride = stride
        self.padding = padding

        # Initialize weights (filters) of size (out_channels, in_channels, kernel_size, kernel_size)
        self.weight = torch.nn.Parameter(torch.randn(out_channels, in_channels, kernel_size, kernel_size))
        # Initialize bias of size (out_channels)
        self.bias = torch.nn.Parameter(torch.randn(out_channels))

    def forward(self, x):
        """
        Perform the forward pass of a convolution layer on a mini-batch x using vectorization

        Args:
```

```

        x (torch.Tensor): mini-batch of images of size (batch_size, in_channels, in_height, in_width)

Returns:
    torch.Tensor: output of the convolutional layer of size (batch_size, out_channels, out_height, out_w
    """
    batch_size, in_channels, in_height, in_width = x.size()

    # Calculate output dimensions
    out_height = (in_height + 2 * self.padding - self.kernel_size) // self.stride + 1
    out_width = (in_width + 2 * self.padding - self.kernel_size) // self.stride + 1

    # Unfold the input tensor into patches
    # The output shape will be (batch_size, in_channels * kernel_size * kernel_size, L)
    # where L is the total number of sliding windows (out_height * out_width)
    unfolded_patches = torch.nn.functional.unfold(x,
                                                  kernel_size=self.kernel_size,
                                                  padding=self.padding,
                                                  stride=self.stride)

    # Reshape the weights for matrix multiplication
    # From (out_channels, in_channels, kernel_size, kernel_size) to (out_channels, in_channels * kernel_size
    kernel_resaped = self.weight.view(self.out_channels, -1)

    # Perform batch matrix multiplication
    # unfolded_patches is (batch_size, C_in * kH * kW, L)
    # kernel_resaped is (C_out, C_in * kH * kW)
    # We need (batch_size, L, C_in * kH * kW) @ (C_in * kH * kW, C_out)
    # Result will be (batch_size, L, C_out)
    output_matmul = torch.matmul(unfolded_patches.permute(0, 2, 1), kernel_resaped.T)

    # Add bias (broadcasted across batch_size and L)
    output_with_bias = output_matmul + self.bias

    # Reshape the output to the final convolutional output shape
    # From (batch_size, L, out_channels) to (batch_size, out_channels, out_height, out_width)
    final_output = output_with_bias.permute(0, 2, 1).view(batch_size, self.out_channels, out_height, out_wid

    return final_output

```

```

# Set random seed
torch.manual_seed(42)

# Create an instance of your custom convolution layer
custom_conv = CustomConv2d(in_channels=1, out_channels=6, kernel_size=5)

# Show output for a single example of the dataset
first_batch = next(iter(test_loader))[0]

# Pick one image in the mini-batch
first_image = first_batch[0]
output_custom_conv = custom_conv(first_image.unsqueeze(0))

# Output sample
print("Output shape:", output_custom_conv.size())
print("Output sample:", output_custom_conv[:, :, 0, 0])

```

```

Output shape: torch.Size([1, 6, 24, 24])
Output sample: tensor([[ -0.3627,  2.7504, -2.1521,  1.8907, -2.4571,  1.1121]],
      grad_fn=<SelectBackward0>)

```

Expected result:

```

Output size    [1, 6, 24, 24]
Sample output  [[-0.3627, 2.7504, -2.1521, 1.8907, -2.4571, 1.1121]]

```

```

# Set random seed
torch.manual_seed(42)

# Create an instance of your custom convolution layer
custom_conv = CustomConv2d(in_channels=1, out_channels=6, kernel_size=5)

# Compare with pytorch's conv2d Layer
pytorch_conv = torch.nn.Conv2d(in_channels=1, out_channels=6, kernel_size=5)

# Forward pass of both networks
first_batch = next(iter(test_loader))[0]

# Measure time
t1 = time.perf_counter()
output_custom_conv = custom_conv(first_batch)
t2 = time.perf_counter()

```

```
print(f"Time taken by custom conv: {t2-t1} seconds")

t1 = time.perf_counter()
output_pytorch_conv = pytorch_conv(first_batch)
t2 = time.perf_counter()

print(f"Time taken by pytorch conv: {t2-t1} seconds")
```

```
Time taken by custom conv: 0.0261856639999678 seconds
Time taken by pytorch conv: 0.007214682000039829 seconds
```

Expected result:

```
Time taken by custom conv  0.0031 seconds
Time taken by pytorch conv  0.0008 seconds
```

2.3 MaxPool Layer

Now that you have a vectorized convolution layer, follow the same idea to implement a custom max pool layer. Remember that this layer does not have weights. You can use the `torch.max` function to pool out the maximum values after generating the convolution image chunks.

```
class CustomMaxPool2d(torch.nn.Module):
    def __init__(self, kernel_size, stride=None, padding=0):
        """
        Initialize the hyperparameters of the max pooling layer

        Args:
            kernel_size (int): size of the kernel
            stride (int): stride size
            padding (int): padding size
        """
        super(CustomMaxPool2d, self).__init__()
        self.kernel_size = kernel_size
        self.stride = stride if stride is not None else kernel_size
        self.padding = padding

    def forward(self, x):
        """
        Perform the max pooling operation on a mini-batch x

        Args:
            x (torch.Tensor): mini-batch of images of size (batch_size, in_channels, in_height, in_width)

        Returns:
            torch.Tensor: output of the max pooling layer of size (batch_size, in_channels, out_height, out_width)
        """
        batch_size, in_channels, in_height, in_width = x.size()

        # Calculate output dimensions
        out_height = (in_height + 2 * self.padding - self.kernel_size) // self.stride + 1
        out_width = (in_width + 2 * self.padding - self.kernel_size) // self.stride + 1

        # Unfold the input tensor into patches
        # The output shape will be (batch_size, in_channels * kernel_size * kernel_size, L)
        # where L is the total number of sliding windows (out_height * out_width)
        unfolded_patches = torch.nn.functional.unfold(x,
                                                       kernel_size=self.kernel_size,
                                                       padding=self.padding,
                                                       stride=self.stride)

        # Reshape the unfolded patches to facilitate max operation
        # From (batch_size, in_channels * kernel_size * kernel_size, L)
        # To (batch_size, in_channels, kernel_size * kernel_size, L)
        reshaped_patches = unfolded_patches.view(batch_size, in_channels, self.kernel_size * self.kernel_size, -1)

        # Take the maximum value along the kernel_size * kernel_size dimension
        # The result will be (batch_size, in_channels, L)
        max_pooled_output, _ = torch.max(reshaped_patches, dim=2)

        # Reshape the output to the final max pooling output shape
        # From (batch_size, in_channels, L) to (batch_size, in_channels, out_height, out_width)
        final_output = max_pooled_output.view(batch_size, in_channels, out_height, out_width)

        return final_output

# Set random seed
torch.manual_seed(42)
```

```
# Create an instance of your custom max pooling layer
custom_maxpool = CustomMaxPool2d(kernel_size=2, stride=2)

# Pass the output of the convolutional layer through the pooling layer
output_custom_maxpool = custom_maxpool(output_custom_conv)

# Output sample
print("Output shape:", output_custom_maxpool.size())
print("Output sample:", output_custom_maxpool[0,:,0,0])

Output shape: torch.Size([16, 6, 12, 12])
Output sample: tensor([-0.3627,  2.7504, -2.1521,  1.8907, -2.4571,  1.1121],
      grad_fn=<SelectBackward0>)
```

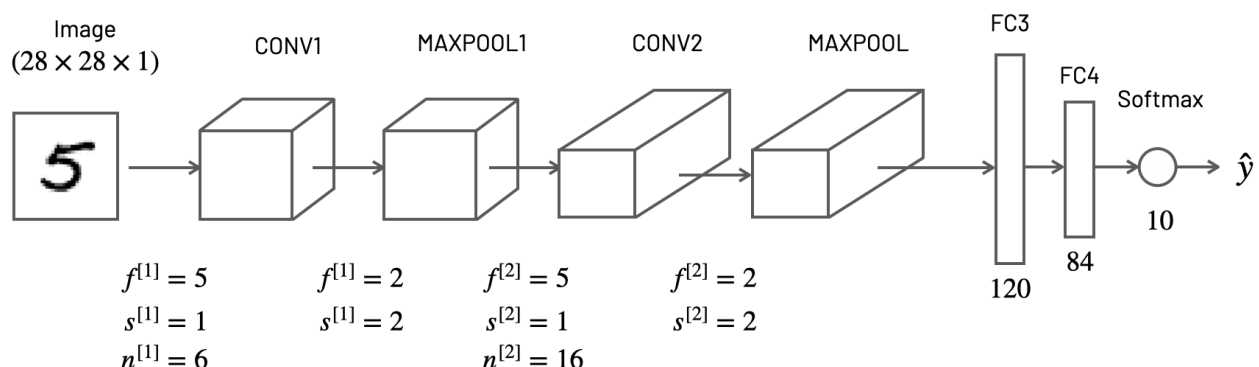
Expected result:

```
Output size      [16, 6, 12, 12]
Sample output    [-0.3627, 2.7504, -2.1521, 1.8907, -2.4571, 1.1121]
```

2.4 LeNet-5

In this section, you will use your custom convolution and pooling layers to implement the variation of LeNet-5 we saw in class (see image below). Similarly to implementing layers, in Pytorch you can create entire models by extending the `torch.nn.module`. You just have to instantiate your layers as attributes in the `__init__` function and implement the full forward pass in the `forward(x)` function. In the forward function, you will pass the input x through the sequence of layers following the specification of the network, which will produce a final prediction $\hat{y}$ at the end. Your forward function has to return this prediction $\hat{y}$. Use the pytorch `torch.nn.functional.relu` function to add the ReLU non-linearity to your convolutional hidden layers.

In Pytorch, it is good practice to not add the softmax (or logistic) classification head as part of the model, since Pytorch can optimize training by computing the softmax/logistic activation as part of the loss function.



```
class LeNet5(torch.nn.Module):
    def __init__(self):
        """
        Initializes the LeNet-5 layers.
        """
        super(LeNet5, self).__init__()
        # C1: Convolutional layer with 6 output channels, 5x5 kernel
        self.conv1 = CustomConv2d(in_channels=1, out_channels=6, kernel_size=5)
        # S2: Max Pooling layer with 2x2 kernel and stride of 2
        self.pool1 = CustomMaxPool2d(kernel_size=2, stride=2)
        # C3: Convolutional layer with 16 output channels, 5x5 kernel
        self.conv2 = CustomConv2d(in_channels=6, out_channels=16, kernel_size=5)
        # S4: Max Pooling layer with 2x2 kernel and stride of 2
        self.pool2 = CustomMaxPool2d(kernel_size=2, stride=2)
        # F5: Fully connected layer with 120 output features
        # The input size to this layer is 16 * 4 * 4 = 256 (from the output of pool2)
        self.fc1 = torch.nn.Linear(16 * 4 * 4, 120)
        # F6: Fully connected layer with 84 output features
        self.fc2 = torch.nn.Linear(120, 84)
        # Output layer: Fully connected layer with 10 output features (for 10 classes)
        self.fc3 = torch.nn.Linear(84, 10)

    def forward(self, x):
        """
        Run the forward pass of the LeNet-5

        Args:
            x (torch.Tensor): mini-batch of size (batch_size, in_channels, in_height, in_width)
        """
```

```

Returns:
    torch.Tensor: logits (ouput of the last layer without softmax) of size (batch_size, 10)
    """
    # Layer C1 -> ReLU -> S2
    x = self.pool1(torch.nn.functional.relu(self.conv1(x)))
    # Layer C3 -> ReLU -> S4
    x = self.pool2(torch.nn.functional.relu(self.conv2(x)))
    # Flatten the output for the fully connected layers
    x = x.view(-1, 16 * 4 * 4) # -1 infers the batch size
    # Layer F5 -> ReLU
    x = torch.nn.functional.relu(self.fc1(x))
    # Layer F6 -> ReLU
    x = torch.nn.functional.relu(self.fc2(x))
    # Output layer
    x = self.fc3(x)
    return x

```

```

# Set random seed
torch.manual_seed(42)

# Create an instance of your LeNet5 model
model = LeNet5()

# Select a single example from the test dataset
example_image, example_label = next(iter(test_loader))
example_image = example_image[0] # Take the first image from the batch
example_label = example_label[0]

# Pass the example through the model
output = model(example_image.unsqueeze(0)) # Add a batch dimension

# Get the predicted class
predicted_class = torch.argmax(output).item()

# Print the results
print(f"Example Image Label: {example_label}")
print(f"Predicted Class: {predicted_class}")

# You can also print the output probabilities if needed
print(f"Output Probabilities: {torch.nn.functional.softmax(output, dim=1)}")

```

```

Example Image Label: 7
Predicted Class: 5
Output Probabilities: tensor([[4.4402e-06, 4.9075e-10, 2.3634e-02, 1.0730e-10, 1.3650e-07, 9.6639e-01,
        2.9831e-03, 9.2747e-04, 6.0459e-03, 1.3411e-05]],
        grad_fn=<SoftmaxBackward0>)

```

Expected result:

```

Example Image Label    7
Predicted Class        1
Output Probabilities    [[1.4715e-07, 9.9989e-01, 2.4902e-12, 2.5233e-07, 1.5861e-06, 8.2712e-10, 5.6732e-07, 1.4339e-11, 1.0663e-04, 7.9855e-11]]

```

Part 3: Training

Now that you have your LeNet-5 defined, it's time to train it with mini-batch gradient descent. Fill out the `train` function above specifying an Adam [optimizer](#), a Categorical CrossEntropy [loss function](#) (also called criterion). Then write the loop to iterate on the number of epochs and, inside the epochs loop, write the loop to iterate on the number of batches. In Pytorch, the dataloader is an iterator, so you can iterate on each mini-batch with `for (x_t,y_t) in train_loader`. For each batch:

1. Reset the gradients with the [torch.optim.Optimizer.zero_grad](#).
2. Run forward pass for the current mini-batch x^t to get the prediction y^t
3. Compute the loss with the `criterion` object
4. Run backward pass from the loss value computed in the previous step (3.)
5. Update the weights of the model with the [torch.optim.Optimizer.step](#)
6. Print the average loss after each 100 mini-batches

```

def train(model, train_loader, epochs=5, lr=0.001):
    """
    Train the model with the Adam optimizer on the training set

    Args:
        model (torch.nn.Module): the model to train
        train_loader (torch.utils.data.DataLoader): the training set
        epochs (int): number of epochs
    """

```



```

lr (float): learning rate

Returns:
    losses (list) : list of average losses after every 100 batches
"""
model.train()

# Define optimizer and loss function
optimizer = torch.optim.Adam(model.parameters(), lr=lr)
criterion = torch.nn.CrossEntropyLoss()

losses = []
for epoch in range(epochs):
    running_loss = 0.0
    for i, (images, labels) in enumerate(train_loader):
        # Zero the parameter gradients
        optimizer.zero_grad()

        # Forward pass
        outputs = model(images)
        loss = criterion(outputs, labels)

        # Backward pass and optimize
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

    # Print statistics every 100 mini-batches
    if (i + 1) % 100 == 0:
        avg_loss = running_loss / 100
        print(f'Epoch [{epoch+1}/{epochs}], Step [{i+1}/{len(train_loader)}], Loss: {avg_loss:.4f}')
        losses.append(avg_loss)
        running_loss = 0.0
    return losses

```

```

# Set random seed
torch.manual_seed(42)

# Instantiate and train the model
model = LeNet5()
losses = train(model, train_loader, epochs=5)

```

```

Epoch [1/5], Step [100/3750], Loss: 1.6982
Epoch [1/5], Step [200/3750], Loss: 0.5936
Epoch [1/5], Step [300/3750], Loss: 0.4285
Epoch [1/5], Step [400/3750], Loss: 0.3964
Epoch [1/5], Step [500/3750], Loss: 0.3396
Epoch [1/5], Step [600/3750], Loss: 0.3232
Epoch [1/5], Step [700/3750], Loss: 0.3672
Epoch [1/5], Step [800/3750], Loss: 0.3621
Epoch [1/5], Step [900/3750], Loss: 0.2528
Epoch [1/5], Step [1000/3750], Loss: 0.2666
Epoch [1/5], Step [1100/3750], Loss: 0.2264
Epoch [1/5], Step [1200/3750], Loss: 0.2287
Epoch [1/5], Step [1300/3750], Loss: 0.3116
Epoch [1/5], Step [1400/3750], Loss: 0.2116
Epoch [1/5], Step [1500/3750], Loss: 0.1986
Epoch [1/5], Step [1600/3750], Loss: 0.2287
Epoch [1/5], Step [1700/3750], Loss: 0.1628
Epoch [1/5], Step [1800/3750], Loss: 0.2274
Epoch [1/5], Step [1900/3750], Loss: 0.1452
Epoch [1/5], Step [2000/3750], Loss: 0.1899
Epoch [1/5], Step [2100/3750], Loss: 0.1886
Epoch [1/5], Step [2200/3750], Loss: 0.1847
Epoch [1/5], Step [2300/3750], Loss: 0.1828
Epoch [1/5], Step [2400/3750], Loss: 0.1798
Epoch [1/5], Step [2500/3750], Loss: 0.1517
Epoch [1/5], Step [2600/3750], Loss: 0.1461
Epoch [1/5], Step [2700/3750], Loss: 0.1373
Epoch [1/5], Step [2800/3750], Loss: 0.1513
Epoch [1/5], Step [2900/3750], Loss: 0.1148
Epoch [1/5], Step [3000/3750], Loss: 0.1401
Epoch [1/5], Step [3100/3750], Loss: 0.1463
Epoch [1/5], Step [3200/3750], Loss: 0.1220
Epoch [1/5], Step [3300/3750], Loss: 0.1150
Epoch [1/5], Step [3400/3750], Loss: 0.1653
Epoch [1/5], Step [3500/3750], Loss: 0.1397
Epoch [1/5], Step [3600/3750], Loss: 0.1455
Epoch [1/5], Step [3700/3750], Loss: 0.1237
Epoch [2/5], Step [100/3750], Loss: 0.0908
Epoch [2/5], Step [200/3750], Loss: 0.0860
Epoch [2/5], Step [300/3750], Loss: 0.0869
Epoch [2/5], Step [400/3750], Loss: 0.1205
Epoch [2/5], Step [500/3750], Loss: 0.0970

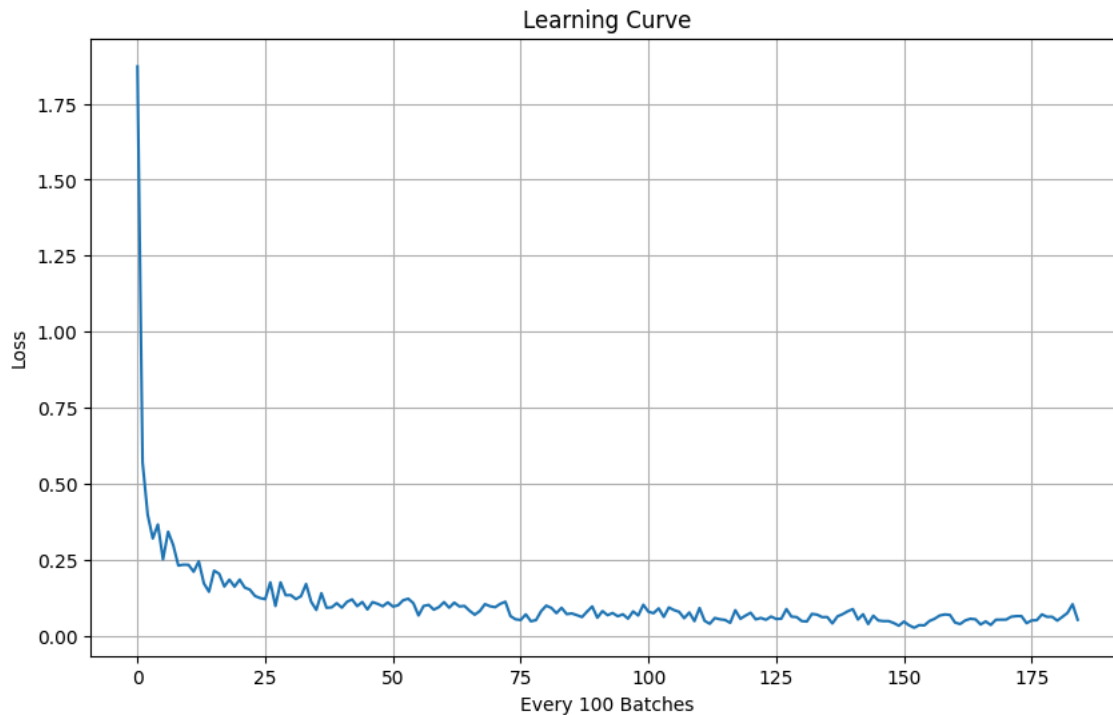
```

```

Epoch [2/5], Step [600/3750], Loss: 0.1139
Epoch [2/5], Step [700/3750], Loss: 0.1058
Epoch [2/5], Step [800/3750], Loss: 0.1079
Epoch [2/5], Step [900/3750], Loss: 0.1156
Epoch [2/5], Step [1000/3750], Loss: 0.1045
Epoch [2/5], Step [1100/3750], Loss: 0.1333
Epoch [2/5], Step [1200/3750], Loss: 0.0943
Epoch [2/5], Step [1300/3750], Loss: 0.1046
Epoch [2/5], Step [1400/3750], Loss: 0.1273
Epoch [2/5], Step [1500/3750], Loss: 0.1050
Epoch [2/5], Step [1600/3750], Loss: 0.1206
Epoch [2/5], Step [1700/3750], Loss: 0.0933
Epoch [2/5], Step [1800/3750], Loss: 0.1206
Epoch [2/5], Step [1900/3750], Loss: 0.1190
Epoch [2/5], Step [2000/3750], Loss: 0.1012
Epoch [2/5], Step [2100/3750], Loss: 0.1000

```

Expected result:



Part 4: Evaluation

The last step is to evaluate your model. The MNIST is a balanced dataset and all classes have the same importance, so we can use accuracy for this task. Implement the following function to compute the accuracy of your LeNet-5 on the MNIST test set.

```

def evaluate(model, test_loader):
    """
    Compute the accuracy of the model on the test set.

    Args:
        model (torch.nn.Module): The model to evaluate.
        test_loader (torch.utils.data.DataLoader): The data loader for the test set.

    Returns:
        float: The accuracy of the model on the test set.
    """
    model.eval() # Set the model to evaluation mode
    correct = 0
    total = 0
    with torch.no_grad(): # Disable gradient calculation during evaluation
        for images, labels in test_loader:
            outputs = model(images)
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    accuracy = 100 * correct / total
    return accuracy

```

```

# Evaluate the model
accuracy = evaluate(model, test_loader)
print(f'Accuracy on test set: {accuracy:.2f}%')

```

Accuracy on test set: 98.22%

Expected result:

Accuracy on test set 98.09%

▾ Part 5: Challenge (Optional)

If you implemented your model correctly, it should have achieved an accuracy higher than 95%. Can you improve the network to do better? Write a model in PyTorch trying to improve the test accuracy. The best model will get an extra point! You can try any idea we've seen in the course so far, such as increasing the model, different pooling layers, regularization (L1, L2, Dropout), etc. Report the results of your model using the `evaluate` function above.

```
class CustomConv2d(torch.nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size, stride=1, padding=0):
        """
        Initialize weights, bias, and hyperparameters of the convolutional layer

        Args:
            in_channels (int): number of input channels
            out_channels (int): number of output channels
            kernel_size (int): size of the kernel
            stride (int): stride size
            padding (int): padding size
        """
        super(CustomConv2d, self).__init__()
        self.in_channels = in_channels
        self.out_channels = out_channels
        self.kernel_size = kernel_size
        self.stride = stride
        self.padding = padding

        # Initialize weights (filters) of size (out_channels, in_channels, kernel_size, kernel_size)
        # Use Kaiming (He) uniform initialization for ReLU activations
        self.weight = torch.nn.Parameter(torch.empty(out_channels, in_channels, kernel_size, kernel_size))
        torch.nn.init.kaiming_uniform_(self.weight, nonlinearity='relu')

        # Initialize bias of size (out_channels) to zeros
        self.bias = torch.nn.Parameter(torch.empty(out_channels))
        torch.nn.init.zeros_(self.bias)

    def forward(self, x):
        """
        Perform the forward pass of a convolution layer on a mini-batch x using vectorization

        Args:
            x (torch.Tensor): mini-batch of images of size (batch_size, in_channels, in_height, in_width)

        Returns:
            torch.Tensor: output of the convolutional layer of size (batch_size, out_channels, out_height, out_width)
        """
        batch_size, in_channels, in_height, in_width = x.size()

        # Calculate output dimensions
        out_height = (in_height + 2 * self.padding - self.kernel_size) // self.stride + 1
        out_width = (in_width + 2 * self.padding - self.kernel_size) // self.stride + 1

        # Unfold the input tensor into patches
        # The output shape will be (batch_size, in_channels * kernel_size * kernel_size, L)
        # where L is the total number of sliding windows (out_height * out_width)
        unfolded_patches = torch.nn.functional.unfold(x,
                                                    kernel_size=self.kernel_size,
                                                    padding=self.padding,
                                                    stride=self.stride)

        # Reshape the weights for matrix multiplication
        # From (out_channels, in_channels, kernel_size, kernel_size) to (out_channels, in_channels * kernel_size * kernel_size)
        kernel_resaped = self.weight.view(self.out_channels, -1)

        # Perform batch matrix multiplication
        # unfolded_patches is (batch_size, C_in * kH * kW, L)
        # kernel_resaped is (C_out, C_in * kH * kW)
        # We need (batch_size, L, C_in * kH * kW) @ (C_in * kH * kW, C_out)
        # Result will be (batch_size, L, C_out)
        output_matmul = torch.matmul(unfolded_patches.permute(0, 2, 1), kernel_resaped.T)

        # Add bias (broadcasted across batch_size and L)
        output_with_bias = output_matmul + self.bias
```

```
# Reshape the output to the final convolutional output shape
# From (batch_size, L, out_channels) to (batch_size, out_channels, out_height, out_width)
final_output = output_with_bias.permute(0, 2, 1).view(batch_size, self.out_channels, out_height, out_width)

return final_output
```

```
class OptionalModel(torch.nn.Module):
    def __init__(self):
        super(OptionalModel, self).__init__()
        # First convolutional block: Conv1 -> ReLU -> MaxPool1
        self.conv1 = CustomConv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1)
        self.pool1 = CustomMaxPool2d(kernel_size=2, stride=2)

        # Second convolutional block: Conv2 -> ReLU -> MaxPool2
        self.conv2 = CustomConv2d(in_channels=32, out_channels=64, kernel_size=3, padding=1)
        self.pool2 = CustomMaxPool2d(kernel_size=2, stride=2)

        # Third convolutional block: Conv3 -> ReLU -> MaxPool3
        # Input to conv3 will be 7x7 (after two 2x2 pools on 28x28: 28/2/2 = 7)
        # By adding padding=1 to pool3, a 7x7 input with kernel_size=2, stride=2
        # and padding=1 results in (7 + 2*1 - 2)/2 + 1 = 4x4 output. ([4.5] = 4)
        self.conv3 = CustomConv2d(in_channels=64, out_channels=128, kernel_size=3, padding=1)
        self.pool3 = CustomMaxPool2d(kernel_size=2, stride=2, padding=1) # Modificado: adicionado padding=1

        # Fully connected layers with dropout
        # The output of pool3 will now be 128 channels * 4x4 feature map (anteriormente 3x3)
        self.fc1 = torch.nn.Linear(128 * 4 * 4, 512) # Modificado: tamanho de entrada para fc1 atualizado para 1
        self.dropout1 = torch.nn.Dropout(0.5)
        self.fc2 = torch.nn.Linear(512, 256)
        self.dropout2 = torch.nn.Dropout(0.5)
        self.fc3 = torch.nn.Linear(256, 10)

    def forward(self, x):
        # First block
        x = self.pool1(torch.nn.functional.relu(self.conv1(x)))
        # Second block
        x = self.pool2(torch.nn.functional.relu(self.conv2(x)))
        # Third block
        x = self.pool3(torch.nn.functional.relu(self.conv3(x)))

        # Flatten the tensor for the fully connected layers
        x = x.view(-1, 128 * 4 * 4) # Modificado: dimensão de flatten atualizada para 128 * 4 * 4

        # Fully connected layers with dropout and ReLU
        x = self.dropout1(torch.nn.functional.relu(self.fc1(x)))
        x = self.dropout2(torch.nn.functional.relu(self.fc2(x)))
        x = self.fc3(x)
        return x
```

```
torch.manual_seed(42)

# Instantiate the optional model
optional_model = OptionalModel()

# Train the optional model
print("\nTraining OptionalModel:")
optional_losses = train(optional_model, train_loader, epochs=5)

# Evaluate the optional model
optional_accuracy = evaluate(optional_model, test_loader)
print(f'\nOptionalModel Accuracy on test set: {optional_accuracy:.2f}%')
```

```
Epoch [5/5], Step [200/3750], Loss: 0.0534
Epoch [5/5], Step [300/3750], Loss: 0.0509
Epoch [5/5], Step [400/3750], Loss: 0.0545
Epoch [5/5], Step [500/3750], Loss: 0.0396
Epoch [5/5], Step [600/3750], Loss: 0.0496
Epoch [5/5], Step [700/3750], Loss: 0.0445
Epoch [5/5], Step [800/3750], Loss: 0.0505
Epoch [5/5], Step [900/3750], Loss: 0.0240
Epoch [5/5], Step [1000/3750], Loss: 0.0377
Epoch [5/5], Step [1100/3750], Loss: 0.0751
Epoch [5/5], Step [1200/3750], Loss: 0.0351
Epoch [5/5], Step [1300/3750], Loss: 0.0488
Epoch [5/5], Step [1400/3750], Loss: 0.0863
Epoch [5/5], Step [1500/3750], Loss: 0.0812
Epoch [5/5], Step [1600/3750], Loss: 0.0459
Epoch [5/5], Step [1700/3750], Loss: 0.0685
Epoch [5/5], Step [1800/3750], Loss: 0.0584
Epoch [5/5], Step [1900/3750], Loss: 0.0569
Epoch [5/5], Step [2000/3750], Loss: 0.0715
Epoch [5/5], Step [2100/3750], Loss: 0.0833
Epoch [5/5], Step [2200/3750], Loss: 0.0570
Epoch [5/5], Step [2300/3750], Loss: 0.0510
Epoch [5/5], Step [2400/3750], Loss: 0.0386
Epoch [5/5], Step [2500/3750], Loss: 0.0410
Epoch [5/5], Step [2600/3750], Loss: 0.0558
Epoch [5/5], Step [2700/3750], Loss: 0.0459
Epoch [5/5], Step [2800/3750], Loss: 0.0833
Epoch [5/5], Step [2900/3750], Loss: 0.0555
Epoch [5/5], Step [3000/3750], Loss: 0.0691
Epoch [5/5], Step [3100/3750], Loss: 0.0576
Epoch [5/5], Step [3200/3750], Loss: 0.0625
Epoch [5/5], Step [3300/3750], Loss: 0.0553
Epoch [5/5], Step [3400/3750], Loss: 0.0303
Epoch [5/5], Step [3500/3750], Loss: 0.0421
Epoch [5/5], Step [3600/3750], Loss: 0.0564
Epoch [5/5], Step [3700/3750], Loss: 0.0222
```

OptionalModel Accuracy on test set: 99.07%

Explain how you changed your model: ...

O que mudei:

Inicialização do CustomConv2d:

A inicialização dos pesos para CustomConv2d foi alterada para a inicialização uniforme de Kaiming (`torch.nn.init.kaiming_uniform_`). Esta é uma prática comum para camadas seguidas por ativações ReLU, pois ajuda a manter uma variância consistente das ativações em toda a rede, evitando problemas como gradientes vanishing ou exploding em modelos mais profundos. Os vieses foram explicitamente inicializados para zeros (`torch.nn.init.zeros_`). Arquitetura do OptionalModel (célula 83c1e15b):

Ajuste de Pooling:

Na terceira camada de max-pooling (`self.pool3`), foi adicionado `padding=1`. Esta modificação ajuda a reter mais informações espaciais nos mapas de características, evitando que eles diminuam muito rapidamente. Essencialmente, ele adiciona uma borda de zeros ao redor da entrada antes do pooling, o que pode ser benéfico ao lidar com mapas de características menores. Entrada da Camada Totalmente Conectada: Devido à mudança no pooling (adicionar preenchimento a `pool3`), o tamanho da saída dos blocos convolucionais mudou. Consequentemente, o tamanho da entrada para a primeira camada totalmente conectada (`self.fc1`) foi atualizado de $128 * 3 * 3$ para $128 * 4 * 4$ para corresponder corretamente às novas dimensões do mapa de características achatado.

Comece a programar ou [gere código](#) com IA.